

What is the Term–Document Matrix?

A **Term–Document Matrix (TDM)** is a table that shows which words (terms) occur in which documents.

- Each **row** → represents a word (term)
- Each **column** → represents a document (Doc1, Doc2, ...)
- Each **cell (value)** →
 - **1** → the term appears in that document
 - **0** → the term doesn't appear

It's the **most basic representation** of text in Information Retrieval (IR).

Example

Suppose we have 3 short documents:

Document	Text
Doc1	I have fever and headache
Doc2	He has fever and cough
Doc3	Cough and headache are common

After tokenizing, removing stopwords, and stemming → we have:

Term	Doc1	Doc2	Doc3
fever	1	1	0
headache	1	0	1
cough	0	1	1
common	0	0	1

This means:

- “fever” appears in Doc1 and Doc2
- “headache” appears in Doc1 and Doc3
- “cough” appears in Doc2 and Doc3
- “common” appears in Doc3 only

How Search Works with TDM

If the user types:

“fever **AND** cough”

We find the rows for both words:

```
fever    → 110
cough    → 011
AND      → 010
```

So only **Doc2** contains both “fever” and “cough”.

That’s how Boolean search works in a Term–Document Matrix.

Problem with TDM

The main problem: it becomes **very large and mostly empty**.

If we have:

- 1,000,000 documents
- 200,000 unique words

Then the matrix has **200 billion cells**,
and most of them are zeros because not every word appears in every document.

This wastes:

- **Memory space.**
- **Search time.**

So, we need something **smarter**.

The Solution — Inverted Index

Instead of mapping “which words are in each document” like TDM,
we map “**which documents contain each word**”.

That’s why it’s called **inverted index** — it reverses the direction.

Example

Term	Documents
fever	Doc1, Doc2
headache	Doc1, Doc3
cough	Doc2, Doc3
common	Doc3

Now, to find documents with “fever AND cough”,
we just take the intersection of their lists:

```
fever → [Doc1, Doc2]
cough → [Doc2, Doc3]
Intersection → [Doc2]
```

Result: **Doc2**

Why Inverted Index is Better

Feature	Term–Document Matrix	Inverted Index
Structure	Table (terms × documents)	Dictionary (term → document list)
Size	Very large	Compact مضغوط
Search speed	Slow	Fast
Data	Mostly zeros	Only stores matches
Use	Research, analysis	Real search engines (Google, Elasticsearch, etc.)

Example in Real Life

When you search on Google for “flu symptoms”,
Google doesn’t read every webpage.
It just looks in its **inverted index**, finds all pages containing those words,
and quickly returns the intersection.

That’s how search engines work behind the scenes

Python Example

```
docs = {
    "Doc1": "fever headache",
    "Doc2": "fever cough",
    "Doc3": "cough headache"
}

# Term-Document Matrix
terms = sorted(set(" ".join(docs.values()).split()))
tdm = {term: [1 if term in docs[d] else 0 for d in docs] for term in terms}

print("TDM:")
print(f"{'Term':<10}{'Doc1':<6}{'Doc2':<6}{'Doc3':<6}")
for t, v in tdm.items():
    print(f"{t:<10}{v[0]:<6}{v[1]:<6}{v[2]:<6}")

# Inverted Index
inverted = {}
for d, text in docs.items():
    for word in text.split():
        inverted.setdefault(word, []).append(d)

print("\nInverted Index:")
for term, files in inverted.items():
    print(f"{term:<10} → {' ', ' '.join(files)}")
```

Summary in Simple English

Concept	TDM	Inverted Index
Direction	Document → Words	Word → Documents
Search time	Slow	Fast
Size	Huge	Small
Used in	Research	Real IR systems
Example	fever → 110	fever → [Doc1, Doc2]

TDM = a big attendance sheet showing which student attended each class.

Inverted Index = a notebook showing which students attended *Math*, *Science*, etc.

The second one is smaller and faster to use.